

Enunciado TPE Final

Trabajo Práctico Especial - Entrega Final

Objetivo

Iterar sobre la aplicación creada durante la cursada y dejarla productiva en un servidor web sin pérdida de datos desde la segunda entrega.

Hacer uso de **Jersey** para la creación de una API REST que sirva a una single page application con **AngularJS** (u otro framework equivalente a elección), implementando todas las estrategias de mejora de tiempos de carga vistas en clase.

Tecnología a utilizar

- La aplicación debe poseer una arquitectura según **Domain Driven Design**, aplicando el patrón de inyección de dependencias a través de **Spring**
- Se deben escribir test unitarios que den cobertura suficiente al sistema, tanto en el backend como en el frontend.
- El frontend debe ser exclusivamente utilizando **AngularJS** (o framework equivalente) contra una REST API creada con **Jersey**. El uso de vistas o controllers de Spring está prohibido.
- La configuración de seguridad deberá realizarse usando **Spring Security**
- Para la persistencia de los objetos de dominio se debe utilizar **Hibernate** como proveedor de **JPA**
- Para la administración y construcción del proyecto se debe utilizar **Maven**. Ejecutar *únicamente* `mvn package` debe ser suficiente para reproducir el war completo utilizado.
- La aplicación debe almacenar los datos en una base de datos **PostgreSQL**.
- El versionado de código debe realizarse mediante el repositorio **GIT** provisto por la cátedra.

Enunciado

Los alumnos deben reemplazar el uso de spring webmvc de las iteraciones anteriores en favor de **AngularJS** (o equivalente) y **Jersey** sin pérdida de funcionalidad ni información en el servidor. La aplicación web debe estar optimizada para dar buenos tiempos de carga, haciendo uso de las múltiples técnicas vistas en clase:

- lazy loading
- minificación
- cache busting + cache incondicional
- cache condicional

Adicionalmente se espera que los alumnos sigan puliendo la funcionalidad conforme el feedback recibido en las iteraciones anteriores para maximizar el valor aportado al usuario. No se requiere funcionalidad nueva salvo que la corrección de la segunda entrega cuestione la falta de funcionalidad.

Finalmente, se espera que los alumnos resuelvan **todos** los problemas relevados por la cátedra en la evaluación de la primer entrega.

Entrega

La totalidad del código debe estar disponible en el repositorio git provisto por la cátedra. Se espera que el mismo **no incluya** archivos innecesarios al estarse usando **Maven**, como ser archivos de configuración de Eclipse, dependencias o archivos generados por **Maven** mismo.

Se espera que el proyecto pueda producir un **war** usable con sólo utilizar `mvn clean package` y que al mismo ser deployado en un **Application Container** existiendo una base de datos **PostgreSQL** con permisos adecuados, se encargue de generar todas las tablas necesarias para ser utilizado.

Adicionalmente, la aplicación debe estar funcionando correctamente en el servidor provisto por la cátedra.

En caso de que la aplicación requiera algún tipo de autenticación para accederla, debe existir un archivo `README.md` que indique claramente credenciales para al menos un usuario con cada nivel de acceso existente en la aplicación.

La entrega se realizará **mediante un email a la cátedra con el hash al commit deployado** a más tardar el día y horario establecido para el final, acompañado de una demo online (via Google Meet o equivalente) con la cátedra.

Evaluación

Se evaluará el grado de cumplimiento de los requerimientos y la calidad del desarrollo (funcionamiento, usabilidad, diseño de objetos).

La cátedra hará una corrección no sólo funcional, sino que también auditará código, analizará commits y contribuciones individuales de cada miembro del equipo. Así, la cátedra se reserva el derecho de evaluar en forma individual a un alumno si lo considera necesario.

Se evaluará

- Apropiado nivel de Unit Testing y calidad de los tests unitarios. Cabe destacar que en esta entrega habrá mucha lógica client-side, que debe estar apropiadamente testada. Su incumplimiento se penalizará con hasta 1 punto.
- Forma de entrega acorde a lo enunciado. Su incumplimiento se penalizará con hasta 1 punto.
- Funcionalidad y usabilidad del producto entregado. Se podrá penalizar hasta 4 puntos por fallos en este frente.
- Correcta arquitectura e implementación. Se podrá penalizar hasta 4 puntos por problemas derivados de no seguir Domain Driven Design, no hacer una correcta separación de capas y responsabilidades o no seguir Effective Java u otras buenas prácticas de programación.
- Pérdidas de información. Se penalizarán con 2 puntos.
- Corrección de feedback de entregas anteriores. Se penalizará con hasta 1 punto *por cada corrección no resuelta*.

Errores no recuperables

Se considerará reprobado sin posibilidad de recuperar el trabajo práctico en caso de:

- No presentarse a la demo online.

- Indicar un hash de un commit posterior a la fecha de inicio del examen.
- No cumplir con los requerimientos técnicos establecidos en el enunciado.
- Inconsistencias en el hash del commit presentado.
- Si la API presentada no sigue los lineamientos de REST / HATEOAS vistos en clase.

Bibliografía

Se recuerda a los alumnos la **fuerte recomendación** de leer:

- Effective Java, 3rd Edition de Joshua Block
- Domain Driven Design, de Evans